

Smart Lender: AI-Powered Loan Eligibility Prediction System

Project Description:

Smart Lender is a Machine Learning-powered web application that automates the loan approval and rejection process by predicting an applicant's loan eligibility based on their financial and personal information. Traditional loan approval methods involve manual verification, making the process time-consuming, inconsistent, and susceptible to human bias. Smart Lender addresses these challenges by providing accurate and real-time loan eligibility predictions using a trained Random Forest Classifier.

The platform features a Loan Prediction Module, where users enter applicant details such as income, education, employment status, loan amount, credit history, and property area. The application preprocesses the input data, applies the trained machine learning model, and instantly predicts whether the loan is Approved or Rejected. The system also incorporates automatic data preprocessing, missing value handling, feature encoding, and model persistence to ensure reliable and consistent predictions.

Built using Python, Flask, HTML, CSS, and Scikit-learn, Smart Lender provides a simple, responsive, and user-friendly web interface. The application uses a pre-trained Random Forest model stored with Joblib, enabling fast prediction with an average inference time of less than one millisecond. By automating loan eligibility assessment, Smart Lender helps financial institutions improve efficiency, reduce manual effort, and support consistent decision-making.

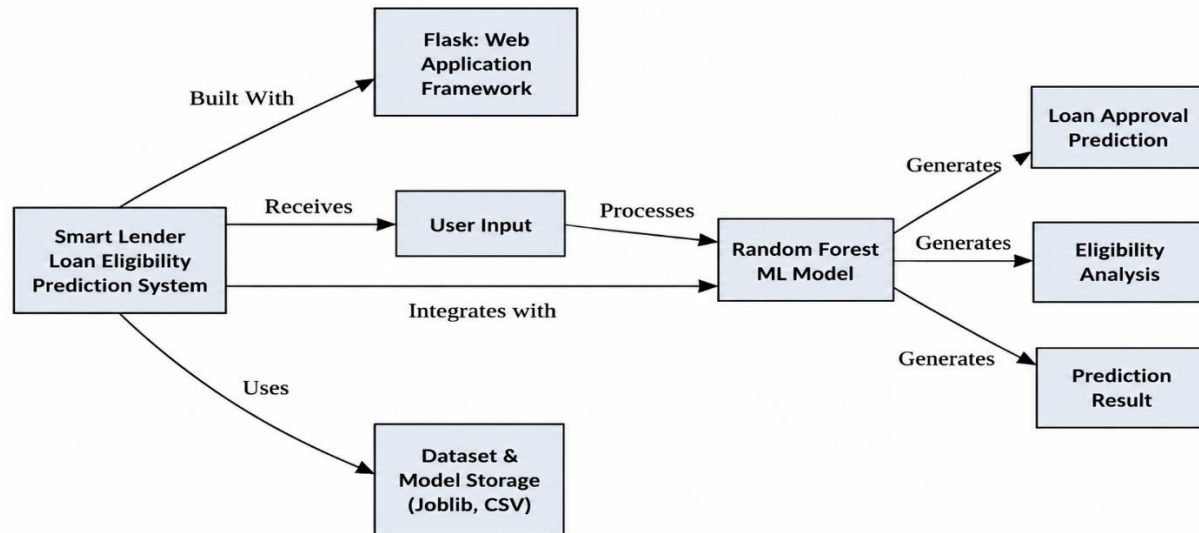
Scenarios

Scenario 1: A customer applies for a personal loan by entering details such as monthly income, loan amount, education level, employment status, and credit history through the Smart Lender web application. The system processes the information using the trained Random Forest model and instantly predicts that the applicant is **Eligible for Loan Approval**, allowing the customer to receive an immediate decision.

Scenario 2: A bank employee receives a loan application from a customer with moderate income and multiple dependents. After entering the applicant's information into Smart Lender, the system evaluates all relevant financial factors and predicts **Loan Rejected** due to insufficient eligibility based on the trained machine learning model, enabling faster and unbiased decision-making.

Scenario 3: A financial institution processes multiple loan applications throughout the day using Smart Lender. Each applicant's information is automatically preprocessed and evaluated by the machine learning model, providing instant approval or rejection decisions within milliseconds. This significantly reduces manual verification time, improves operational efficiency, and ensures consistent loan eligibility assessment across all applications.

Technical Architecture:



Description: Smart Lender follows a modular three-tier architecture to provide fast and reliable loan eligibility prediction. The Presentation Layer consists of a responsive web interface developed using HTML5, CSS3, and Jinja2 templates, where users enter applicant details. The Application Layer, built with the Flask framework, validates user inputs, preprocesses applicant data, and communicates with the trained Random Forest Machine Learning model to generate loan eligibility predictions. The Data Layer stores the trained machine learning model using Joblib along with the loan dataset in CSV format, ensuring efficient model loading and real-time prediction. This architecture provides accurate, scalable, and user-friendly loan eligibility assessment for applicants and financial institutions.

(Note: This project uses CSV files and serialized machine learning models (Joblib) for data storage instead of a relational database. Therefore, an ER Diagram is not applicable)

Pre-requisites:

- **Python Programming Knowledge:** Python Documentation
- **Flask Framework Knowledge:** Flask Documentation
- **Machine Learning Fundamentals:** Scikit-learn Documentation
- **Data Analysis using Pandas and NumPy:** Pandas & NumPy Documentation
- **HTML, CSS, and Jinja2 Templates:** W3Schools HTML/CSS Tutorials
- **Model Serialization with Joblib:** Joblib Documentation
- **Version Control using Git and GitHub:** Git Documentation
- **Development Environment Setup:** Flask Installation Guide

Project Workflow:

Milestone 1: Dataset Collection and Model Development

- **Activity 1.1:** Collect and analyze the loan eligibility dataset from Kaggle/UCI repository.
- **Activity 1.2:** Perform Exploratory Data Analysis (EDA) to understand feature relationships and identify missing values.
- **Activity 1.3:** Clean and preprocess the dataset by handling missing values, encoding categorical variables, and preparing features for training.
- **Activity 1.4:** Train and evaluate multiple machine learning classification models and select the Random Forest Classifier based on prediction accuracy.

Milestone 2: Machine Learning Model Integration

- **Activity 2.1:** Serialize the trained Random Forest model using Joblib for efficient loading during prediction.
- **Activity 2.2:** Develop the prediction module to preprocess user inputs and generate real-time loan eligibility predictions.

Milestone 3: Flask Application Development

- **Activity 3.1:** Develop the main application logic in **app.py** using the Flask framework.
- **Activity 3.2:** Implement application routing, input validation, prediction processing, and result generation.

Milestone 4: Frontend Development

- **Activity 4.1:** Design and develop a responsive user interface using **HTML5**, **CSS3**, and **Jinja2 Templates**.
- **Activity 4.2:** Create dynamic forms for applicant information and display loan eligibility prediction results in an intuitive format..

Milestone 5: Testing and Deployment

- **Activity 5.1:** Perform functional testing to validate prediction accuracy, user input handling, and application workflow.
- **Activity 5.2:** Deploy the Flask application on Render and verify accessibility, performance, and end-to-end functionality.

Milestone 6: Documentation and Conclusion

- **Activity 6.1:** Prepare comprehensive project documentation, including system architecture, workflow diagrams, testing reports, and user guides.
- **Activity 6.2:** Summarize project outcomes, evaluate model performance, identify limitations, and recommend future enhancements such as user authentication, cloud database integration, and support for additional machine learning models.

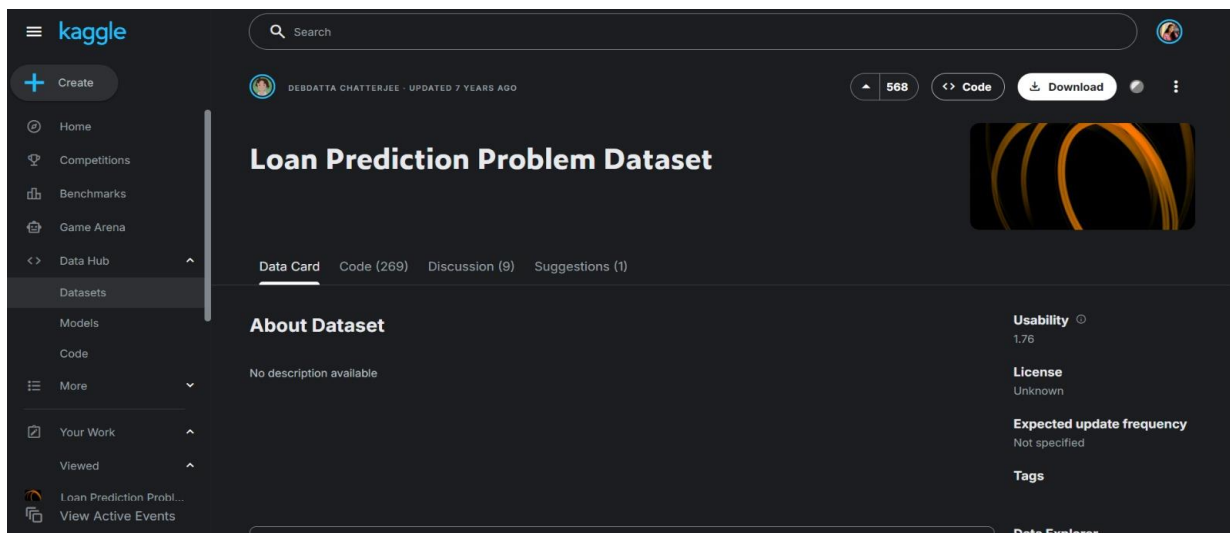
Milestone 1: Machine Learning Model Development and Architecture

In this milestone, the primary objective is to develop a robust machine learning model capable of accurately predicting loan eligibility. This involves collecting and analyzing the loan dataset, preprocessing the data, selecting the most suitable classification algorithm, and designing the overall system architecture. The architecture defines how the user interface, Flask backend, machine learning model, and data storage components interact to provide real-time loan eligibility predictions.

Activity 1.1: Collect and Prepare the Loan Eligibility Dataset

- Before training the machine learning model, a reliable dataset must be collected and prepared. Follow the steps below to preprocess the dataset for model development.

- **Step 1** — Download the Loan Eligibility Prediction Dataset from a trusted source such as the Kaggle or UCI Machine Learning Repository. The dataset contains applicant information including income, education, employment status, loan amount, credit history, and loan approval status.



Step 2 — Load the Dataset: Load the dataset into Python using the Pandas library for further analysis and preprocessing.

```
# Load dataset
loan_data= pd.read_csv("../data/train_u6lujuX_CVtuZ9i.csv")

# View first rows
loan_data.head()
```

	Loan_ID	Gender	Married	Dependents	Education	Self_Employed	ApplicantIncome	CoapplicantIncome	LoanAmount	Loan_Amount_Term	Credit_History	Property_A
0	LP001002	Male	No	0	Graduate	No	5849	0.0	NaN	360.0	1.0	Ur
1	LP001003	Male	Yes	1	Graduate	No	4583	1508.0	128.0	360.0	1.0	R
2	LP001005	Male	Yes	0	Graduate	Yes	3000	0.0	66.0	360.0	1.0	Ur
3	LP001006	Male	Yes	0	Not Graduate	No	2583	2358.0	120.0	360.0	1.0	Ur
4	LP001008	Male	No	0	Graduate	No	6000	0.0	141.0	360.0	1.0	Ur

- **Step 3 — Explore the Dataset:** Analyze the dataset by checking the data types, summary statistics, and identifying missing values.

	ApplicantIncome	CoapplicantIncome	LoanAmount	Loan_Amount_Term	Credit_History
count	614.000000	614.000000	592.000000	600.000000	564.000000
mean	5403.459283	1621.245798	146.412162	342.000000	0.842199
std	6109.041673	2926.248369	85.587325	65.12041	0.364878
min	150.000000	0.000000	9.000000	12.000000	0.000000
25%	2877.500000	0.000000	100.000000	360.000000	1.000000
50%	3812.500000	1188.500000	128.000000	360.000000	1.000000
75%	5795.000000	2297.250000	168.000000	360.000000	1.000000
max	81000.000000	41667.000000	700.000000	480.000000	1.000000

- **Step 4 — Data Preprocessing:**

Prepare the dataset by:

- ☐ Handling missing values.
- ☐ Encoding categorical variables.
- ☐ Removing unnecessary columns.
- ☐ Splitting features and target variables.

- **Step 5 — Train the Machine Learning Model**

- Train multiple classification algorithms such as:
 - Decision Tree
 - Logistic Regression
 - Random Forest
 - XGBoost
- Evaluate each model using performance metrics such as accuracy, precision, recall, and F1-score. Select the Random Forest Classifier as the final model based on its superior prediction performance.

Step 6 — Save the Trained Model

Serialize the trained Random Forest model using Joblib so that it can be loaded efficiently by the Flask application during prediction.

Step 7 — Integrate the Model with Flask

Load the trained model into the Flask application to perform real-time loan eligibility predictions.

Important — Protect Model and Dataset Files

Store the trained model and dataset securely within the project structure. Exclude unnecessary files such as virtual environments, cache folders, and temporary files by adding them to the .gitignore file.

```
__pycache__/  
venv/  
*.pyc  
.env
```

Activity 1.2: Research and Select the Appropriate Machine Learning Model

The following research activities were performed to identify the most suitable machine learning model for predicting loan eligibility:

- Understand the Project Requirements: Analyze the project objective of predicting loan approval based on applicant information such as income, education, employment status, loan amount, credit history, and property area.
- Study the Loan Eligibility Dataset: Examine the dataset structure, identify numerical and categorical features, detect missing values, and understand the relationship between input features and the target variable.
- Evaluate Multiple Machine Learning Models: Train and compare various classification algorithms, including Logistic Regression, Decision Tree, Random Forest, and XGBoost, using evaluation metrics such as accuracy, precision, recall, and F1-score.
- Select the Optimal Model: Choose the Random Forest Classifier because it achieved the highest prediction accuracy, handled mixed data types effectively, minimized overfitting, and provided reliable performance for loan eligibility prediction. The trained model is serialized using Joblib for efficient deployment within the Flask application.

Activity 1.3: Define the Architecture of the Application

- Draft an Architectural Diagram: Create a layered architecture illustrating the interaction between the Presentation Layer (HTML, CSS, Jinja Templates), Flask Application Layer, Machine Learning Prediction Module, and Data Storage Layer.
- Define Frontend Functionality: Design a responsive web interface that enables users to enter applicant details, including gender, marital status, education, employment status, applicant income, co-applicant income, loan amount, loan term, credit history, and property area, and view the loan eligibility prediction.
- Outline Backend Responsibilities: Develop the Flask backend to receive HTTP requests, validate and preprocess user inputs, invoke the trained Random Forest model, generate loan eligibility predictions, and return the prediction results to the user interface.
- Describe Machine Learning Integration: Integrate the serialized Random Forest Classifier with the Flask application using Joblib. The backend preprocesses incoming data, performs feature encoding where required, loads the trained model, executes real-time prediction, and displays whether the applicant is Eligible or Not Eligible for loan approval.

Activity 1.4: Set Up the Development Environment

- **Install Python and Pip:** Ensure Python 3.8+ is installed on your machine along with pip for dependency management.
- **Create a Virtual Environment:** Set up a virtual environment using venv to isolate project dependencies:

```
D:\projects> python -m venv  
venvD:\projects> "d:\projects\venv\Scripts\activate"  
(myenv) D:\projects> pip install -r requirements.txt
```

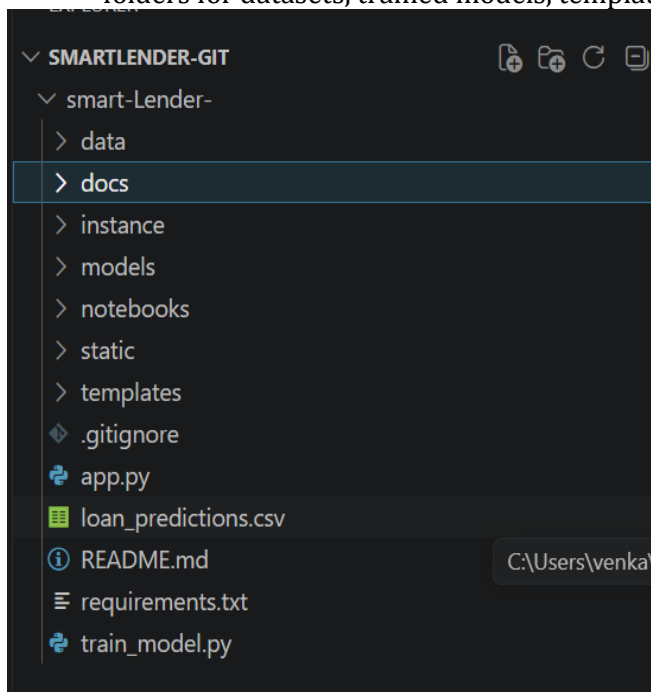
- **Install Flask:** Use pip to install Flask:

```
(myenv) D:\projects> pip install Flask==3.0.3
```

- **Install Required Machine Learning Libraries:** Install all the necessary libraries required for data preprocessing, model training, and prediction.

```
(myenv) D:\projects> pip install scikit-learn pandas numpy joblib xgboost
```

- **Configure the Project Environment:** Create the project directory structure, including folders for datasets, trained models, templates, static files, and documentation.



Milestone 2: Core Functionalities Development

Activity 2.1: Develop Core Loan Eligibility Prediction Features

Implement the core functionalities required for accurate and real-time loan eligibility prediction using machine learning.

- Applicant Information Collection: Develop a user-friendly input form that collects applicant details such as Gender, Marital Status, Education, Self-Employment Status, Applicant Income, Co-applicant Income, Loan Amount, Loan Amount Term, Credit History, and Property Area.
 - Data Preprocessing Module: Preprocess user inputs by handling missing values, encoding categorical features, validating input data, and converting the information into the format required by the trained machine learning model.
 - Loan Eligibility Prediction Module: Use the trained Random Forest Classifier to analyze the processed applicant information and predict whether the applicant is Eligible or Not Eligible for loan approval with high accuracy.
-

Activity 2.2: Implement the Flask Backend

Define Routes in Flask

- Configure the necessary routes in app.py to serve the homepage, receive applicant information, and display loan eligibility prediction results.
-

Process User Input

- Develop an input form in index.html that allows users to enter all required loan applicant information.
 - Use Flask's request.form to retrieve submitted form data from the frontend.
 - Validate all mandatory fields and ensure that numerical values such as income and loan amount are entered correctly before processing.
 - Perform feature encoding and preprocessing to convert user input into a format compatible with the trained Random Forest model.
-

Integrate the Machine Learning Model

- Load the serialized Random Forest Classifier using the Joblib library during application startup.
- Pass the preprocessed applicant information to the trained model for prediction.
- Generate a real-time loan eligibility prediction indicating whether the applicant is Eligible or Not Eligible for loan approval.
- Return the prediction result to the frontend and display it through a clean and user-friendly interface.

Milestone 3: App.py Development

Milestone 3 focuses on developing and refining the core logic of the app.py file, which serves as the backbone of the Smart Lender application. During this milestone, the Flask application is configured to handle user requests, process applicant information, interact with the trained machine learning model, and generate real-time loan eligibility predictions. Unit testing is also performed to ensure reliable application functionality and accurate prediction results.

Activity 3.1: Writing the Main Application Logic in app.

Define the Core Routes in app.py:

Establish the primary routes for the Smart Lender application, where each route performs a specific function:

- / — Home page route that renders the main index.html interface, allowing users to enter loan applicant information.
- /predict — POST endpoint that receives applicant details submitted through the web form, preprocesses the input data, invokes the trained Random Forest Classifier, and returns the loan eligibility prediction (Eligible or Not Eligible) to the user interface.

```
@app.route("/predict", methods=["POST"])
def predict(): try: gender =
request.form["Gender"] married =
request.form["Married"] dependents =
request.form["Dependents"] education =
request.form["Education"] self_employed =
request.form["Self_Employed"]
applicant_income =
float(request.form["ApplicantIncome"])
coapplicant_income =
float(request.form["CoapplicantIncome"])
loan_amount =
float(request.form["LoanAmount"])
loan_amount_term =
float(request.form["Loan_Amount_Term"])
credit_history =
float(request.form["Credit_History"])
property_area =
request.form["Property_Area"] # Preprocess
user input input_data = preprocess_input(
gender, married, dependents, education,
self_employed, applicant_income,
coapplicant_income, loan_amount,
loan_amount_term, credit_history,
property_area )max_tokens=2048,
```

```
)  
  
# Predict loan eligibility prediction =  
model.predict(input_data) result = "Eligible" if prediction[0]  
== 1 else "Not Eligible" return render_template("result.html",  
prediction=result) except Exception as e: return  
render_template("index.html", error=str(e))
```

Set Up Route Handlers for Each Feature:

- **For the /predict route**, implement logic that retrieves the applicant details submitted through the loan application form using `request.form` (or `request.get_json()` if JSON is used).
- **Apply input validation:** Ensure all mandatory fields are provided and validate numerical inputs such as Applicant Income, Co-applicant Income, Loan Amount, Loan Amount Term, and Credit History. If any required field is missing or contains invalid data, return an appropriate error message to the user.
- **Preprocess the input data:** Convert categorical values into numerical representations, perform feature encoding, and arrange the input features in the format expected by the trained **Random Forest Classifier**.
- **Invoke the Machine Learning model:** Load the serialized Random Forest model using **Joblib**, pass the processed applicant information to the model, and generate the loan eligibility prediction.
- **Return the prediction result:** Display whether the applicant is **Eligible** or **Not Eligible** for loan approval through the Flask application by rendering the prediction result on the web page.
- **Handle exceptions gracefully:** Capture unexpected errors during preprocessing or prediction and display meaningful error messages instead of allowing the application to crash.

Define Feature Processing Helpers:

- **Feature Encoding Dictionary:** Maps categorical applicant information to numerical values that can be processed by the trained Random Forest Classifier.

```
GENDER = {
    "Male": 1,
    "Female": 0
}

MARRIED = {
    "Yes": 1,
    "No": 0
}

EDUCATION = {
    "Graduate": 1,
    "Not Graduate": 0
}

SELF_EMPLOYED = {
    "Yes": 1,
    "No": 0
}

PROPERTY_AREA = {
    "Urban": 2,
    "Semiurban": 1,
    "Rural": 0
}

LOAN_STATUS = {
    1: "Eligible",
    0: "Not Eligible"
}}
```

FEATURE_VALIDATION dictionary: Defines validation rules for each applicant attribute to ensure that valid data is provided before passing it to the machine learning model.

- **build_prompt():** assembles a structured prompt string from the product info, selected platform tip, and tone description, instructing the model to return strictly valid JSON.

```
dFEATURE_VALIDATION = {
    "ApplicantIncome": "Must be a positive numeric value.",
    "CoapplicantIncome": "Must be zero or a positive numeric value.",
    "LoanAmount": "Must be greater than zero.",
    "Loan_Amount_Term": "Loan repayment period in months.",
    "Credit_History": "Accepts only 0 or 1.",
    "Gender": "Male or Female.",
    "Married": "Yes or No.",
    "Education": "Graduate or Not Graduate.",
    "Self_Employed": "Yes or No.",
    "Property_Area": "Urban, Semiurban, or Rural."
}
```

Integrate the Machine Learning Model in the Prediction Function

- Load the trained Random Forest model using the Joblib library when the Flask application starts to avoid loading the model for every prediction request.
- Receive applicant information from the frontend, preprocess the input data, encode categorical variables, and convert the values into the format expected by the trained machine learning model.
- Pass the processed feature vector to the Random Forest Classifier using the predict() method to determine the applicant's loan eligibility.
- Interpret the prediction result by mapping the model output to a user-friendly response (Eligible or Not Eligible) before displaying it on the frontend.
- Return the prediction result to the user by rendering the result page with the predicted loan eligibility status.

```
def predict_loan(input_data):  
    """  
    Predict loan eligibility using the trained  
    Random Forest Classifier.  
    """  
  
    prediction = model.predict(input_data)  
  
    if prediction[0] == 1:  
        return "Eligible"  
  
    return "Not Eligible"
```

Application Flow

- **Home Route:** / → Renders **index.html**, where users enter applicant information.
- **Prediction Route:** /predict (**POST**) → Receives applicant details, validates and preprocesses the data, invokes the trained Random Forest model, and returns the loan eligibility prediction to the frontend.

Error Handling

- Validate that all required applicant information is provided before prediction.
- Ensure numerical fields contain valid numeric values.
- Handle invalid or missing input gracefully by displaying meaningful error messages.
- Catch unexpected exceptions during preprocessing or prediction to prevent application failure.

Milestone 4: Frontend Development

Milestone 4 focuses on developing a user-friendly and responsive interface for the Smart Lender – Loan Eligibility Prediction System. This involves designing an intuitive web interface using HTML, CSS, and Bootstrap, allowing users to enter loan applicant information easily and view prediction results in a clear and organized manner.

Activity 4.1: Designing and Developing the User Interface

Set Up the Base HTML Structure:

- Develop the main HTML file (**index.html**) as the primary interface, including a project header, loan application form, and prediction result section
- Use semantic HTML elements to organize the layout, improving readability, accessibility, and maintainability.

Design the form to collect all required applicant information, including Gender, Marital Status, Dependents, Education, Self Employment Status, Applicant Income, Co-applicant Income, Loan Amount, Loan Amount Term, Credit History, and Property Area.

Design a Responsive Layout Using CSS:

- Create a CSS stylesheet (**static/css/style.css**) to provide a clean, modern, and responsive interface with consistent colors, typography, spacing, and form styling.
- Use **Bootstrap Grid, Flexbox**, and responsive design techniques to ensure the application functions correctly across desktops, tablets, and mobile devices.
- Apply appropriate styling to buttons, input fields, forms, and result cards to improve user experience and readability.

Create Separate UI Components

- **Loan Application Form:** A structured form containing all applicant information required for loan eligibility prediction.
- **Prediction Result Card:** Displays the predicted loan eligibility status (Eligible or Not Eligible) along with a clear visual indication.
- **Navigation/Header Section:** Provides the project title and brief description for improved usability.
- **Validation Messages:** Display appropriate validation and error messages whenever invalid or incomplete information is submitted.

Activity 4.2: Creating Dynamic Content Rendering

Handle Form Submission

- Capture applicant information entered through the loan application form and submit it securely to the Flask backend using the HTTP POST method.
- Validate user inputs before submitting the form to ensure all required fields are completed correctly.
- Display a loading state while the application processes the prediction request.

Display Prediction Results

- **Receive the loan eligibility prediction from the Flask backend.**
- **Dynamically display the prediction result on the web page without requiring additional user interaction.**
- **Clearly indicate whether the applicant is Eligible or Not Eligible for loan approval.**

Improve User Experience

- **Reset the form after prediction when required.**
- **Display meaningful error messages for invalid or incomplete inputs.**
- **Ensure smooth navigation, responsive layouts, and consistent user interface behavior across different devices and screen sizes.**

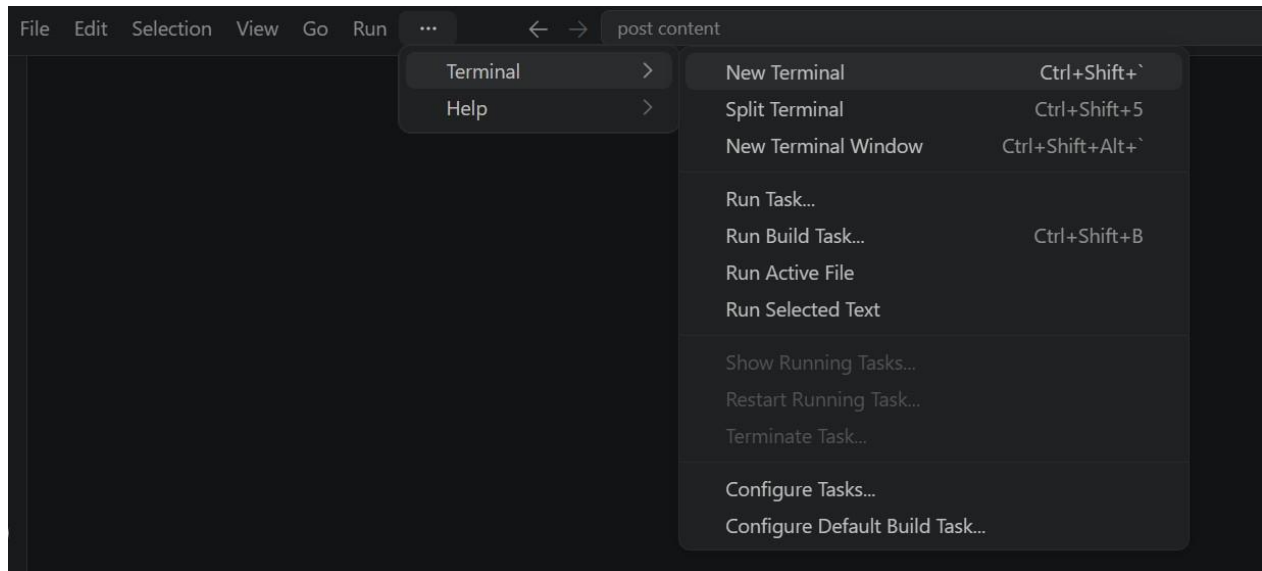
Milestone 5: Deployment

In Milestone 5, the focus is on deploying the CaptionAI application first locally to verify correct operation, and then publicly via Ngrok to share the app over a secure, accessible URL. This involves configuring the server environment, managing dependencies, and launching the app for real-world use.

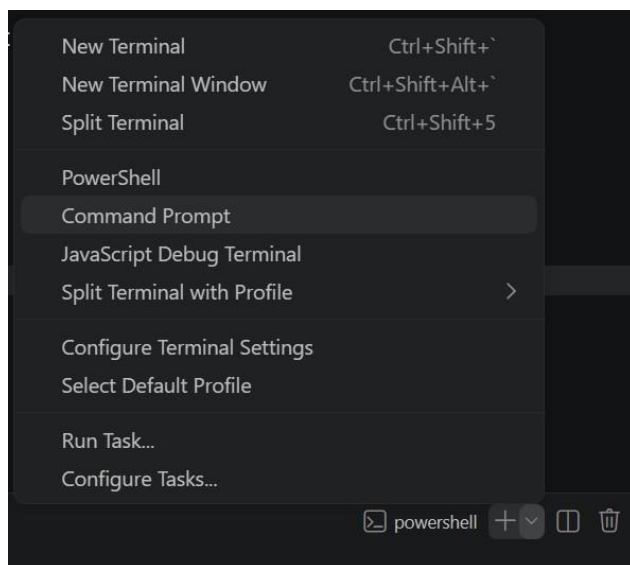
Activity 5.1: Preparing the Application for Local Deployment

Set Up a Virtual Environment:

- Open a New Terminal



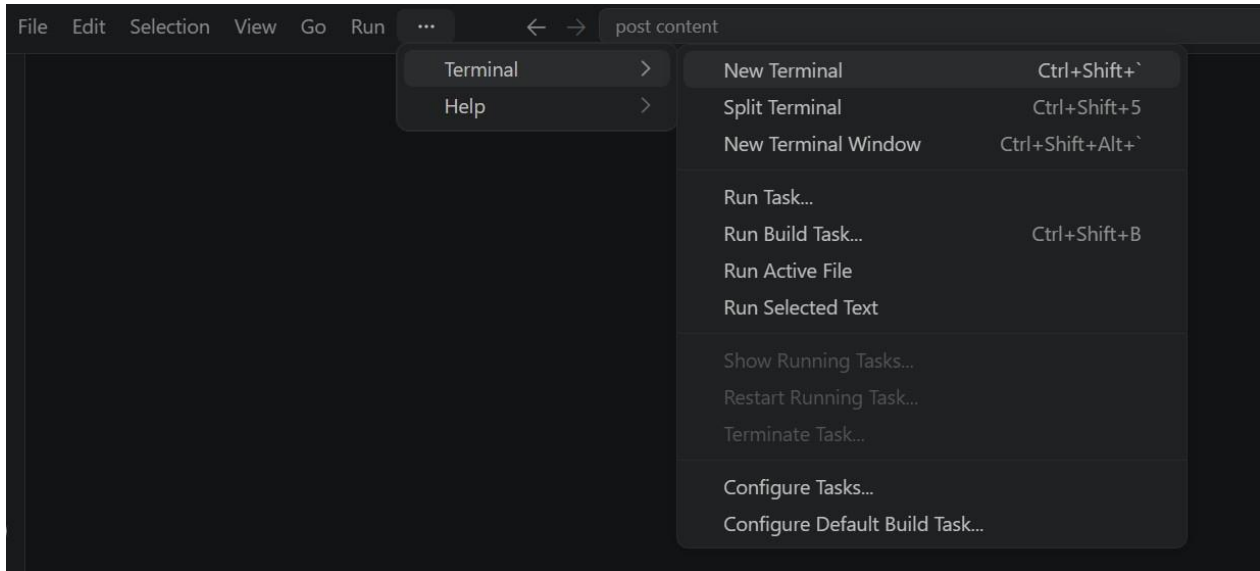
- Then open "Command Prompt"



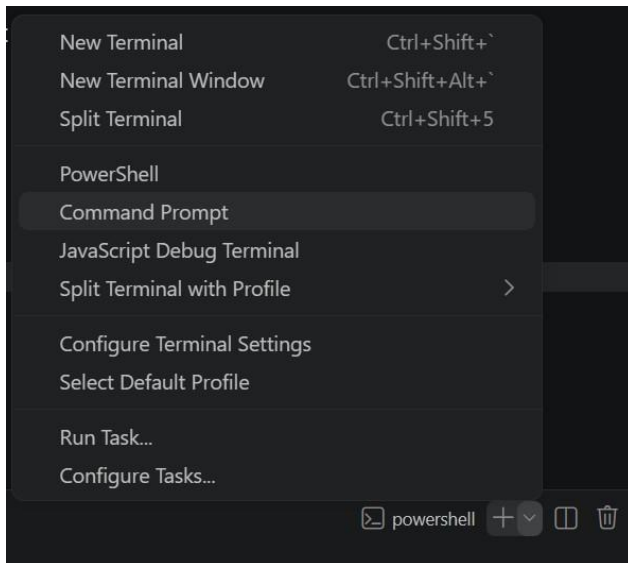
Activity 5.1: Preparing the Application for Local Deployment

Set Up a Virtual Environment:

- Open a New Terminal



- Then open “Command Prompt”



- Create and activate a virtual environment, then install all required dependencies from requirements.

```
D:\projects>python -m venv myenv
D:\projects>"d:\projects\venv\Scripts\activate"
(myenv) D:\projects>pip install -r requirements.txt
```

Configure Environment Variables:

- Create a .env file in the project root and add your Groq API key. The app loads this automatically using python-dotenv at startup:
GROQ_API_KEY=your_groq_api_key_here

Activity 5.2: Local Testing and Verification

Start the Flask Development Server:

- Run the application locally using:

```
(myenv) D:\projects>python app.py
```

- Once running, open a browser and navigate to "<http://127.0.0.1:5000>" to access the app.

Test the Loan Eligibility Prediction System

Perform multiple test cases using different applicant details to verify the application's functionality.

- Enter valid applicant information and submit the loan application.
- Verify that the system correctly predicts whether the applicant is **Eligible** or **Not Eligible** for loan approval.
- Test various scenarios by changing parameters such as **Applicant Income**, **Loan Amount**, **Credit History**, and **Property Area** to ensure the prediction model responds appropriately.
- Verify that invalid or incomplete inputs display appropriate validation messages without causing application errors.
- Confirm that the prediction results are displayed correctly on the result page and that the application performs consistently across different web browsers and devices.

Activity 5.3: Public Deployment via RENDER

The Smart Lender – Loan Eligibility Prediction System is deployed on Render, a cloud application hosting platform that enables the Flask application to be securely accessed through a public URL. Render automatically builds, deploys, and hosts the application, making it available from any device with an internet connection.

Deploy the Application on Render

Create a Render Account

- Sign up or log in to the **Render** platform.
- Connect your GitHub account and authorize access to the Smart Lender project repository.

Create a New Web Service

- Select **New** → **Web Service** from the Render Dashboard.
- Choose the **Smart Lender** GitHub repository.
- Configure the deployment settings including the project name, runtime environment, and branch.

Configure the Build and Start Commands

Specify the build and startup commands for the Flask application.

Build Command

```
D:\projects> pip install -r requirements.txt
```

- **START COMMAND:**

```
D:\projects> python app.py
```

Deploy the Application

- Click the **Deploy Web Service** button.
- Render automatically installs all project dependencies, builds the application, and starts the Flask server.
- Once deployment is complete, Render generates a public URL for accessing the application.

```
=====
YOUR PUBLIC WEBSITE URL IS LIVE!
URL: https://smart-lender-n96s.onrender.com
=====
```

Verify the Public Deployment

- Open the generated Render URL in a web browser.
- Verify that the **Smart Lender – Loan Eligibility Prediction System** loads successfully.
- Enter sample applicant information and confirm that loan eligibility predictions are generated correctly.
- Test the application from multiple browsers and devices to ensure consistent functionality.

Important Notes

- Ensure that all project dependencies are listed correctly in the **requirements.txt** file before deployment.
- Verify that the trained machine learning model and required project files are included in the GitHub repository.
- Every push to the connected GitHub repository automatically triggers a new deployment on Render.
- Monitor deployment logs through the Render Dashboard to identify and resolve any build or runtime issues.

Exploring the Website's Web Pages:

Home / Generator Page:



Description:

The single-page interface of **Smart Lender – Loan Eligibility Prediction System** serves as both the landing page and the loan eligibility prediction platform. It features a clean, responsive, and user-friendly interface developed using **HTML, CSS, Bootstrap, and Flask**, with a well-organized layout that simplifies the loan application process. The page includes a project header displaying the **Smart Lender** title and a brief description of the system. The main section consists of a comprehensive loan application form where users enter applicant details such as personal information, income, loan amount, credit history, and property area. Upon submission, the system processes the input using a trained **Random Forest Machine Learning model** and instantly displays the loan eligibility prediction (**Eligible** or **Not Eligible**) on the same interface, providing users with a seamless and efficient experience across desktop and mobile devices.

Input Section:

Applicant Details

Enter the required financial and personal information below.

Gender	Marital Status
Select Gender	Select Status
Dependents	Education
No. of Dependents	Select Education
Self Employed	Applicant Income (₹)
Select Status	e.g. 50000
Coapplicant Income (₹)	Loan Amount (₹)
e.g. 0	e.g. 150
Loan Term (Days)	Credit History
360 (Default)	Select Credit History
Property Area	
Select Area Type	

Analyze Eligibility

Description A clean and responsive form card that serves as the primary loan application interface. Users enter the required applicant information, including Gender, Marital Status, Dependents, Education, Self-Employment Status, Applicant Income, Co-applicant Income, Loan Amount, Loan Amount Term, Credit History, and Property Area. The form validates the entered information before submission to ensure accuracy and completeness. Once all mandatory fields are provided, users click the Predict Loan Eligibility button to submit the application. While the system processes the request, the application validates the input, performs data preprocessing, invokes the trained Random Forest Machine Learning model, and then displays the loan eligibility prediction (Eligible or Not Eligible) through a simple and user-friendly interface.

Results Section:

Applicant Details

Enter the required financial and personal information below.

Prediction Result
❌ Loan Rejected

Gender	Marital Status
Select Gender	Select Status
Dependents	Education
No. of Dependents	Select Education
Self Employed	Applicant Income (₹)
Select Status	e.g. 50000
Coapplicant Income (₹)	Loan Amount (₹)
e.g. 0	e.g. 150
Loan Term (Days)	Credit History
360 (Default)	Select Credit History
Property Area	
Select Area Type	

Analyze Eligibility

Description:

Displayed immediately after the user submits the loan application, this section presents the **loan eligibility prediction** generated by the trained **Random Forest Machine Learning model**. The prediction result is shown in a dedicated result card, clearly indicating whether the applicant is **Eligible** or **Not Eligible** for loan approval. The page may also display a brief confirmation message along with the submitted applicant details for reference. If invalid or incomplete information is entered, appropriate validation or error messages are presented to guide the user. After processing, the interface automatically updates to display the prediction result, providing a smooth and user-friendly experience without requiring additional navigation.

Conclusion:

Smart Lender – Loan Eligibility Prediction System is a machine learning-based web application developed using **Flask** that simplifies and automates the loan eligibility assessment process. The system leverages a trained **Random Forest Classifier** to analyze applicant information and provide fast, accurate, and reliable loan approval predictions through an intuitive and responsive web interface. The project encompassed the complete machine learning lifecycle, including data collection, preprocessing, model training and evaluation, Flask application development, frontend design, and deployment on **Render**, demonstrating how machine learning can improve decision-making efficiency and reduce manual effort in financial institutions.

Looking ahead, the Smart Lender platform offers significant opportunities for enhancement, such as integrating **user authentication**, connecting with **real-time banking and financial databases**, incorporating **credit score and financial history analysis**, and providing **personalized loan recommendations** based on applicant profiles. Additional improvements, including advanced data visualization dashboards, explainable AI techniques for transparent predictions, and support for multiple loan products, can further enhance the system's usability and scalability. With continuous refinement of the prediction model and application features, Smart Lender has the potential to evolve into a comprehensive intelligent loan management and decision support system for modern financial institutions.

